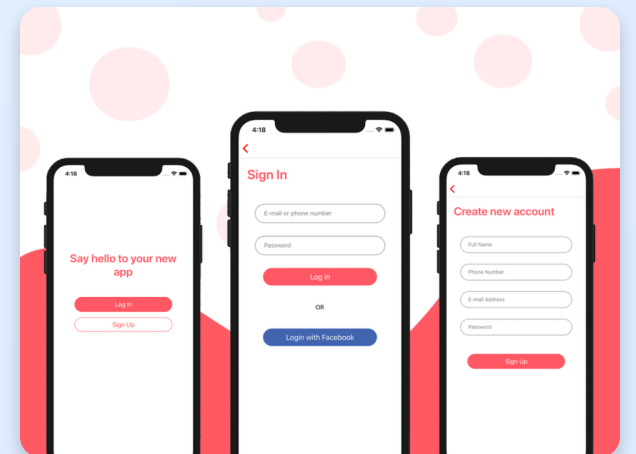


앱프로그래밍 4주차 1차시 (이론)

주제: 이벤트 처리와 폼

담당교수 안효정 | 인공지능학과

2025년 9월 22일



학습 목표

- > React 이벤트 처리 방식 이해
- > onChange, onSubmit 이벤트 동작 원리 학습
- > Controlled Form의 개념 정리
- > To-do List 앱 구현 기초 준비

💡 오늘 배우는 내용은 실제 앱 개발에서 가장 많이 사용하는 입력 처리 방식을 중심으로 진행합니다.

지난 시간 복습

- ✓ **조건부 렌더링:** JSX에서 조건문 활용

```
{isLoggedIn ? : }
```

- ✓ **리스트 렌더링:** map() 함수로 배열 순회, key 속성 필수

```
{items.map(item =>  
  • {item.text}  
)}
```

- ✓ **계산기 앱 완성:** state, 이벤트, 연산 기록 관리

계산기 앱 결과물

128			
7	8	9	÷
4	5	6	×
1	2	3	-
C	0	=	+

연산 기록: 100 + 28 = 128

→ 이번 시간: 폼 입력과 이벤트 흐름

- i 계산기 앱에서 사용한 이벤트 핸들링 기법을 확장하여 다양한 사용자 입력을 처리하는 방법을 배웁니다.

Part A

이벤트 처리 기초 안내

React 이벤트 처리 기본 개념을 배웁니다.



React 이벤트의 특징

- > HTML 이벤트와 유사하지만 `camelCase`로 작성
- > 함수 참조를 전달해야 함 (문자열이 아님)

```
<button onClick={handleClick}>클릭</button>
```

- > 이벤트 핸들러는 모든 브라우저에서 동일하게 작동 (크로스 브라우저 호환)

💡 React는 HTML과 달리 함수 객체를 직접 전달해야 합니다. HTML에서는 `onclick="함수()"` 형식의 문자열을 사용합니다.

SyntheticEvent

React의 이벤트 래퍼(wrapper)

크로스 브라우저 호환성 보장

일반 이벤트 메서드 사용 가능

```
function handleSubmit(e) {  
  // 기본 새로고침 방지  
  e.preventDefault();  
  // 이벤트 전파 중지  
  e.stopPropagation();  
}
```

- ✓ `preventDefault()` 로 브라우저 기본 동작을 방지합니다.
- ✓ `stopPropagation()` 으로 이벤트 버블링을 중단할 수 있습니다.

💡 SyntheticEvent는 모든 브라우저에서 동일한 API를 제공하여 이벤트 처리를 일관되게 만듭니다.

이벤트 핸들러 작성 규칙

함수 이름: 보통 `handleXxx` 형식으로 작성

이벤트 속성에 함수 참조(객체)를 전달해야 함

```
function MyComponent() {  
  const handleClick = () => {  
    console.log('버튼이 클릭되었습니다!');  
  };  
  
  return (  
    <div>  
      <button onClick={handleClick}>OK</button> // ✅ 올바른 방법  
      <button onClick={handleClick()}>OK</button> // ❌ 잘못된 방법  
    </div>  
  );  
}
```

✅ `onClick={handleClick}`: 함수 참조를 전달하여 클릭 시 함수 실행

❌ `onClick={handleClick()}`: 렌더링 시점에 함수가 즉시 실행됨

⚠️ 매개변수 전달 필요 시 화살표 함수로 래핑: `onClick={() => handleClick(id)}`

이벤트 객체 접근

이벤트 핸들러는 이벤트 객체(e)를 매개변수로 받아 발생한 요소에 접근할 수 있습니다.

```
function handleClick(e) {  
  // 이벤트가 발생한 요소 참조  
  console.log("타겟:", e.target);  
  
  // 입력 요소의 값 확인  
  console.log("값:", e.target.value);  
}
```

→ `e.target` : 이벤트 발생 요소 참조

→ `e.target.value` : 입력값

→ `e.target.checked` : 체크박스 상태

💡 이벤트 객체를 통해 폼 요소의 값을 읽고 상태(state)로 관리하면 Controlled Component를 구현할 수 있습니다.

클릭 이벤트 예제



버튼 클릭 시 알림 메시지를 표시하는 이벤트 핸들러를 작성해봅시다.

```
export default function MyButton() {  
  const handleClick = () => alert("버튼 클릭됨!");  
  
  return (  
    <button onClick={handleClick}>눌러보세요</button>  
  );  
}
```

💡 직접 코딩해보고 브라우저에서 alert 동작을 확인해보세요.

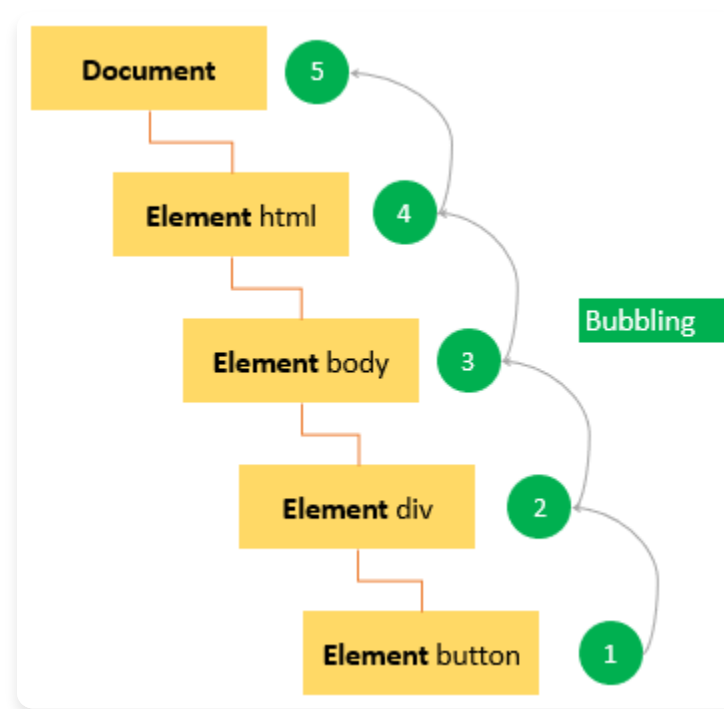
</> 실습하기

코드펜이나 로컬 환경에서 직접 구현해보세요!



이벤트 전파와 성능

- ✓ React는 이벤트를 **루트(root)**에서 위임 처리
- ✓ 모든 이벤트 핸들러를 DOM 요소에 직접 부착하지 않음
- ✓ 단일 이벤트 리스너로 메모리 사용량 감소
- ✓ SyntheticEvent로 브라우저 간 호환성 보장



💡 리액트 17부터 이벤트 위임이 루트 DOM 컨테이너로 변경되어 여러 React 앱을 한 페이지에서 안전하게 중첩할 수 있게 되었습니다.

HTML vs React 이벤트

구분	HTML	React
문법	<code>onclick="..."</code> (문자열)	<code>onClick={함수}</code> (함수 참조)
대소문자	소문자 <code>onclick</code>	camelCase <code>onClick</code>
전달 방식	문자열로 된 코드	함수 객체 참조
이벤트 객체	브라우저별 다른 구현	SyntheticEvent로 통일
코드 예시	<code><button onclick="alert('클릭')"></code>	<code><button onClick={handleClick}></code>

</> HTML 이벤트

- 문자열 기반 이벤트 처리
- 브라우저별 호환성 문제 가능
- 인라인 함수 선언 방식

React 이벤트

- 함수 객체 전달 방식
- SyntheticEvent로 일관된 동작
- 성능 최적화에 유리

React는 이벤트 처리를 위해 HTML과 유사하지만 보다 일관된 방식을 제공합니다.

연습 문제 1



버튼 클릭 시 콘솔에 "클릭됨" 출력하는 이벤트 핸들러를 작성해봅시다.

```
function ClickButton() {  
  return (  
    <button onClick={() => console.log("클릭됨")}>  
      Click  
    </button>  
  );  
}
```

💡 console.log()를 사용해 개발자 도구의 콘솔 창에서 출력을 확인할 수 있습니다.

</> 실습하기

코드를 입력하고 개발자 도구(F12)를 열어 콘솔 탭에서 결과를 확인해보세요.



이벤트 처리 기초 정리

- ✓ React 이벤트 = SyntheticEvent
- ✓ 함수 참조 전달 방식 기억!

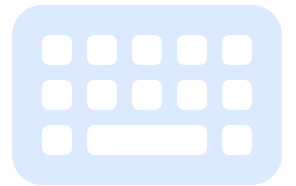
❓ 퀴즈: 다음 중 올바른 React 이벤트 핸들러 전달 방식은?

- 1 `<button onclick="handleClick()">클릭</button>`
- 2 `<button onClick={handleClick}>클릭</button>`
- 3 `<button onClick="handleClick">클릭</button>`

💡 함수를 실행한 결과가 아닌 **함수 자체**를 전달하는 것이 중요합니다!

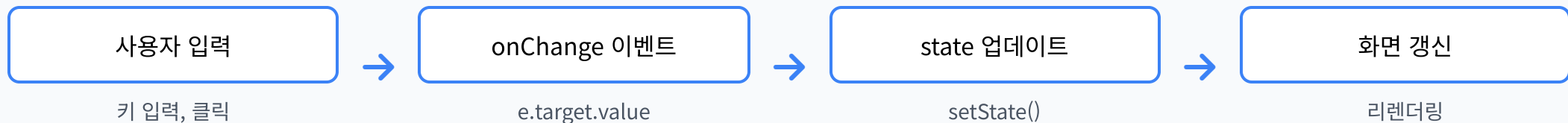
onChange 이벤트 안내

React에서 입력값 변경 처리를 배웁니다.



onChange 이벤트

- 입력값이 변경될 때마다 발생하는 이벤트
- 다양한 입력 요소에서 사용 가능
 - 텍스트 입력 (input, textarea)
 - 체크박스 (input type="checkbox")
 - 라디오 버튼 (input type="radio")
 - 선택 상자 (select)



💡 onChange 이벤트는 React에서 가장 빈번하게 사용되는 이벤트 중 하나로, 실시간 입력 처리의 기본이 됩니다.

onChange 기본 예시

텍스트 입력값을 state로 관리하는 간단한 예시 - 입력할 때마다 state가 갱신됨

```
function TextInput() {  
  const [text, setText] = useState("");  
  return (  
    <input  
      value={text}  
      onChange={(e) => setText(e.target.value)}  
    />  
  );  
}
```

 사용자 키 입력 → onChange 이벤트 발생

 setText() 함수로 상태 업데이트 → 리렌더링

  **입력할 때마다 state가 갱신됩니다.** 이것이 React의 단방향 데이터 흐름입니다.

e.target.value와 checked



텍스트 입력과 체크박스 상태를 React에서 읽고 처리하는 방법

```
function InputDemo() {
  const [text, setText] = useState("");
  const [isChecked, setIsChecked] = useState(false);

  const handleTextChange = (e) => {
    setText(e.target.value); // 텍스트 입력값
  };

  const handleCheckboxChange = (e) => {
    setIsChecked(e.target.checked); // 체크박스 상태
  };

  return (
    <div>
      <input type="text" value={text} onChange={handleTextChange} />
      <div>
        <input type="checkbox" checked={isChecked} onChange={handleCheckboxChange} /> 옵션
      </div>
      <p>입력된 텍스트: {text}</p>
      <p>체크 상태: {isChecked ? "선택됨" : "선택안됨"}</p>
    </div>
  );
}
```

실행 결과:

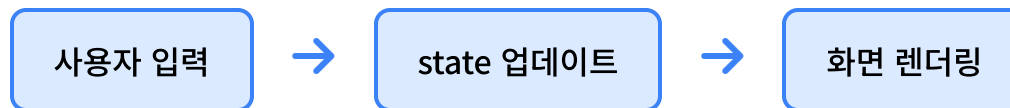
입력된 텍스트: Hello

체크 상태: 선택됨

💡 텍스트 입력은 **e.target.value**로, 체크박스는 **e.target.checked**로 접근

Controlled Input 개념

- > 입력값을 React의 **state**로 관리
- > 단방향 데이터 흐름으로 상태 변화를 예측 가능
- > 폼 제출, 유효성 검사, 조건부 비활성화에 용이



💡 모든 입력 요소의 `value`는 `state`에서 오며, `onChange` 이벤트는 `state`를 갱신합니다. 이것이 React가 권장하는 입력 처리 방식입니다.

연습 문제 2



이름 입력 시 실시간으로 인사말에 표시되는 컴포넌트를 만들어봅시다.

```
import { useState } from 'react';

function GreetingInput() {
  const [name, setName] = useState("");

  return (
    <div>
      <input value={name} onChange={e => setName(e.target.value)}
        placeholder="이름을 입력하세요" />
      <p>안녕하세요, {name ? name : "홍길동"}님!</p>
    </div>
  );
}
```

안녕하세요, 홍길동님!

💡 onChange 이벤트가 발생할 때마다 state가 업데이트되고 컴포넌트가 리렌더링됩니다.



직접 코딩하기

onChange와 state의 관계를 이해해보세요!



onChange·Controlled Input 정리

- ✓ onChange = state 업데이트 트리거
- ✓ Controlled Input = React 기본 패턴
- ✓ 단방향 데이터 흐름으로 예측 가능한 UI 상태 관리

? 퀴즈: Controlled Input의 특징은?

- 1 DOM에서 직접 값을 읽어 관리한다
- 2 React state로 입력값을 관리하고 value 속성에 연결한다
- 3 입력값이 변경될 때마다 페이지가 새로고침된다

💡 **Controlled Input**은 입력값이 React에 의해 완전히 제어되므로 폼 검증과 조건부 처리가 용이합니다.

onSubmit 이벤트 안내

폼 제출 처리에 대해 배웁니다.



onSubmit의 역할

- **폼 제출 시 실행되는 이벤트:** 사용자가 form을 제출할 때 (submit 버튼 클릭 또는 Enter 키) 발생합니다.
- **기본 동작:** 브라우저가 페이지를 새로고침하고 서버에 요청을 보냅니다.
- **React에서 활용:** SPA(Single Page Application)에서는 페이지 새로고침 없이 데이터를 처리해야 합니다.

```
// 폼 제출 핸들러 예시
const handleSubmit = (e) => {
  e.preventDefault(); // 기본 새로고침 방지
  // 폼 데이터 처리 로직
};
```

⚠ **중요:** React에서 폼 제출 처리 시 `e.preventDefault()`를 반드시 호출해야 합니다. 그렇지 않으면 페이지가 새로고침되어 React 상태가 초기화됩니다.

onSubmit 기본 예시

폼 제출을 처리하는 onSubmit 이벤트 핸들러는 폼 데이터를 수집하고 처리합니다.

```
function MyForm() {
  const [name, setName] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault(); // 새로그침 방지
    alert(`제출된 이름: ${name}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={name} onChange={e => setName(e.target.value)} />
      <button type="submit">제출</button>
    </form>
  );
}
```

✓ `e.preventDefault()` : 페이지 새로그침 방지

✓ `type="submit"` 버튼: onSubmit 이벤트 트리거

✓ `useState` : 입력값 상태 관리

✓ `onSubmit`: 사용자 폼 제출 동작 핸들링

💡 폼 제출 처리는 사용자 입력 검증, API 요청, 상태 업데이트 등 다양한 작업을 수행할 수 있습니다.

버튼 타입 주의

⚠ **<button>** 태그의 기본 타입은 `type="submit"` 입니다.

✓ React 폼에서는 명시적으로 `type="button"` 과 `type="submit"` 을 구분해서 사용해야 합니다.

🚩 `type="submit"` (기본값)

폼 제출을 트리거합니다.

```
<button>제출</button>
<button type="submit">제출</button>
```

💡 폼 안에서는 자동으로 `onSubmit` 이벤트를 발생시킵니다.

🖱 `type="button"`

일반 버튼 기능만 수행합니다.

```
<button type="button">클릭</button>
```

💡 폼 제출을 트리거하지 않고 `onClick` 이벤트만 발생시킵니다.

❌ 실수 사례: 폼 내부의 일반 버튼에 타입을 명시하지 않으면 의도치 않은 폼 제출이 발생할 수 있습니다.

💡 **모범 사례:** 버튼의 용도에 따라 항상 명시적으로 `type="button"` 또는 `type="submit"` 을 지정하세요.

연습 문제 3



이메일 입력 폼을 만들고, 제출 시 입력된 이메일 주소를 alert로 표시해보세요.

```
function EmailForm() {
  const [email, setEmail] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault(); // 새로그침 방지
    alert(`입력한 이메일: ${email}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="email" value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="이메일을 입력하세요" />
      <button type="submit">제출</button>
    </form>
  );
}
```

💡 이메일 유효성 검사를 추가하려면 pattern 속성이나 제출 전 정규식 검사를 활용해보세요.

실행 결과: 이메일 제출 시 `alert(`입력한 이메일: user@example.com`)` 형식으로 표시

</> 실습하기

완성 후 다른 유효성 검사도 추가해 보세요!



onSubmit 이벤트 정리

- ✓ onSubmit = 폼 제출 제어
- ✓ e.preventDefault() 필수
- ✓ form 태그 내부의 button은 기본값이 type="submit"

? 퀴즈: 폼 제출 시 페이지 새로고침을 방지하려면?

- 1 return false 사용
- 2 e.preventDefault() 호출
- 3 e.stopPropagation() 호출

💡 React에서 폼 제출 처리는 **항상 기본 동작을 방지**하고 JavaScript로 직접 처리하는 것이 기본입니다!

Controlled vs Uncontrolled Form 안내

Controlled Form과 Uncontrolled Form의 차이와 특징을 배우고, 각 방식의 장단점을 비교합니다.



Controlled Form

- > 입력값을 state로 제어
- > 예측 가능하고 검증/리셋이 용이함
- > React 권장 방식



✔ 모든 폼 데이터는 단일 출처(state)에서 관리되어 신뢰할 수 있는 단일 데이터 소스 유지

💡 폼의 모든 변경사항이 React의 상태 관리 시스템을 통해 처리되므로 데이터 흐름 추적이 용이합니다.

Uncontrolled Form

- > DOM에서 직접 값 읽기 (useRef 등 사용)
- > 단순하지만 상태 관리 어려움
- > 실시간 값 변경 추적 불가능

```
function UncontrolledForm() {  
  const inputRef = useRef();  
  
  const handleSubmit = (e) => {  
    e.preventDefault();  
    console.log(inputRef.current.value);  
  };  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <input ref={inputRef} />  
      <button type="submit">제출</button>  
    </form>  
  );  
}
```

Uncontrolled Form 동작 방식

📄 사용자 입력



</> DOM에 직접 저장 (React state 무시)



🔍 useRef로 필요할 때 DOM에서 값 조회



💡 파일 업로드, 간단한 폼, 라이브러리 통합 시에만 제한적으로 사용합니다. React에서는 **Controlled Form**을 권장합니다.

Controlled vs Uncontrolled 비교 표

방식	특징	장점	단점
Controlled	입력값을 state로 제어	일관성, 검증 용이	코드 다소 복잡
Uncontrolled	DOM에서 직접 값 읽기	단순, 빠른 작성	유지보수 어려움
사용 방식	value + onChange	useRef + defaultValue	

☞ Controlled Form

- React에서 권장하는 기본 방식
- 실시간 입력 상태 확인과 유효성 검증
- 폼 데이터 초기화, 조작이 간편함
- 예측 가능한 단방향 데이터 흐름

</> Uncontrolled Form

- DOM 노드에 직접 접근하는 방식
- 폼 제출 시에만 값을 읽어옴
- 기존 HTML 폼과 유사한 작동 방식
- 간단한 폼에 적합함

React에서는 **Controlled Form**을 사용하는 것이 권장되며, 복잡한 폼 처리와 검증에 더 적합합니다.

Controlled Form 정리

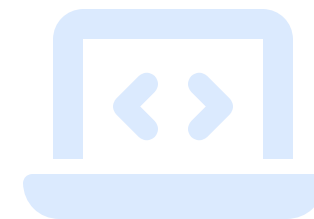
- ✓ 상태(state)로 입력값 제어
- ✓ 데이터 흐름 예측 가능 단방향 데이터 흐름으로 상태 추적이 용이합니다.
- ✓ 유효성 검사(validation) 구현 용이
- ✓ 입력값 초기화 및 조작 간편 상태값만 변경하면 UI가 자동으로 업데이트됩니다.



👉 **Controlled Form**을 기본으로 사용합시다. React에서 권장하는 방식으로 유지보수와 디버깅이 더 쉽습니다.

실습: 간단 로그인 폼

Controlled Form을 활용한 실습 예제



실습: 로그인 폼 동작



ID/비밀번호 입력 → 제출 시 콘솔에 출력

로그인 폼



콘솔 출력 결과

```
> 로그인: "user123" "password123"  
> 입력 데이터 처리 중...
```

폼이 제출되면 `preventDefault()` 를 통해 새로고침을 방지하고
입력된 데이터가 콘솔에 출력됩니다.

```
const handleSubmit = (e) => {  
  e.preventDefault(); // 새로고침 방지  
  console.log("로그인:", id, pw); // 입력값 콘솔에 출력  
};
```

💡 이 실습을 통해 **폼 제출 이벤트**와 **입력값 접근** 방법을 이해할 수 있습니다.

로그인 폼 코드

ID와 비밀번호를 입력받아 처리하는 Controlled Form 컴포넌트

```
function LoginForm() {
  const [id, setId] = useState("");
  const [pw, setPw] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log("로그인:", id, pw);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={id} onChange={e => setId(e.target.value)}
        placeholder="아이디" />
      <input type="password" value={pw}
        onChange={e => setPw(e.target.value)}
        placeholder="비밀번호" />
      <button type="submit">로그인</button>
    </form>
  );
}
```

✓ useState 로 입력값 상태 관리

✓ onSubmit 으로 폼 제출 처리

✓ onChange 로 입력값 변화 감지

✓ preventDefault() 로 새로고침 방지

💡 실제 애플리케이션에서는 로그인 정보를 서버로 전송하거나 상태 관리 라이브러리와 연동합니다.

로그인 폼: 실행 화면

아이디

비밀번호

로그인

입력



제출 (onSubmit)



콘솔 출력

```
> 로그인: "user123" "password123"
```

💡 실제 로그인 폼은 비밀번호 검증, 에러 처리, 서버 통신 등 추가 로직이 포함됩니다.

유효성 검사 추가

비밀번호 길이 < 6 → 경고 메시지 출력
조건부 렌더링을 활용한 간단한 유효성 검사 방법

```
{pw.length < 6 && <p style={{color:"red"}}>비밀번호는 6자 이상</p>}
```

- ✓ 논리 연산자 && 활용 조건부 렌더링
- ✓ 조건 `true` 일 때만 경고 메시지 표시

- ✓ 인라인 스타일로 텍스트 색상 지정
(React에서는 이중 중괄호 사용)

실행 결과 예시:

비밀번호

....

비밀번호는 6자 이상



Controlled Form에서는 실시간으로 입력값 검증이 가능하여 사용자에게 즉각적인 피드백을 제공할 수 있습니다.

- i 유효성 검사는 사용자 경험 향상 뿐만 아니라, 서버 제출 전 데이터 유효성을 확보하는 중요한 과정입니다.

연습 문제 4



이름이 비어 있으면 폼 제출을 막는 유효성 검증을 구현해보세요.

```
function NameForm() {
  const [name, setName] = useState("");
  const [error, setError] = useState("");

  const handleSubmit = (e) => {
    if (name.trim() === "") {
      e.preventDefault();
      setError("이름을 입력해주세요!");
      return;
    }
    setError(""); // 폼 제출 처리
    console.log("제출된 이름:", name);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="이름을 입력하세요" />
      {error && <p style={{color: "red"}}>{error}</p>}
      <button type="submit">제출</button>
    </form>
  );
}
```

💡 입력값이 비어있는지 확인하려면 **trim()** 메서드로 공백을 제거한 후 검사하세요.



실습하기

유효성 검증 로직을 추가해보세요.

Controlled Form과 검증 결합 정리

- ✓ Controlled Form은 상태(state)가 항상 최신 입력값을 반영
- ✓ onChange 이벤트로 실시간 입력값 검증 가능
- ✓ form 제출 시 유효성 검사 로직과 쉽게 결합 가능

</> 유효성 검사 예시

```
// 실시간 유효성 검사
{password.length < 6 && (
  <p style={{color:"red"}}>비밀번호는 6자 이상</p>
)}

// 제출 시 유효성 검사
const handleSubmit = (e) => {
  e.preventDefault();
  if (name.trim() === "") {
    alert("이름을 입력하세요");
    return;
  }
  // 폼 처리 로직...
```



Controlled Form은 모든 입력값이 React 상태로 관리되므로 유효성 검사가 더 쉽고 예측 가능합니다.

To-do List 앱 개념 설계 안내

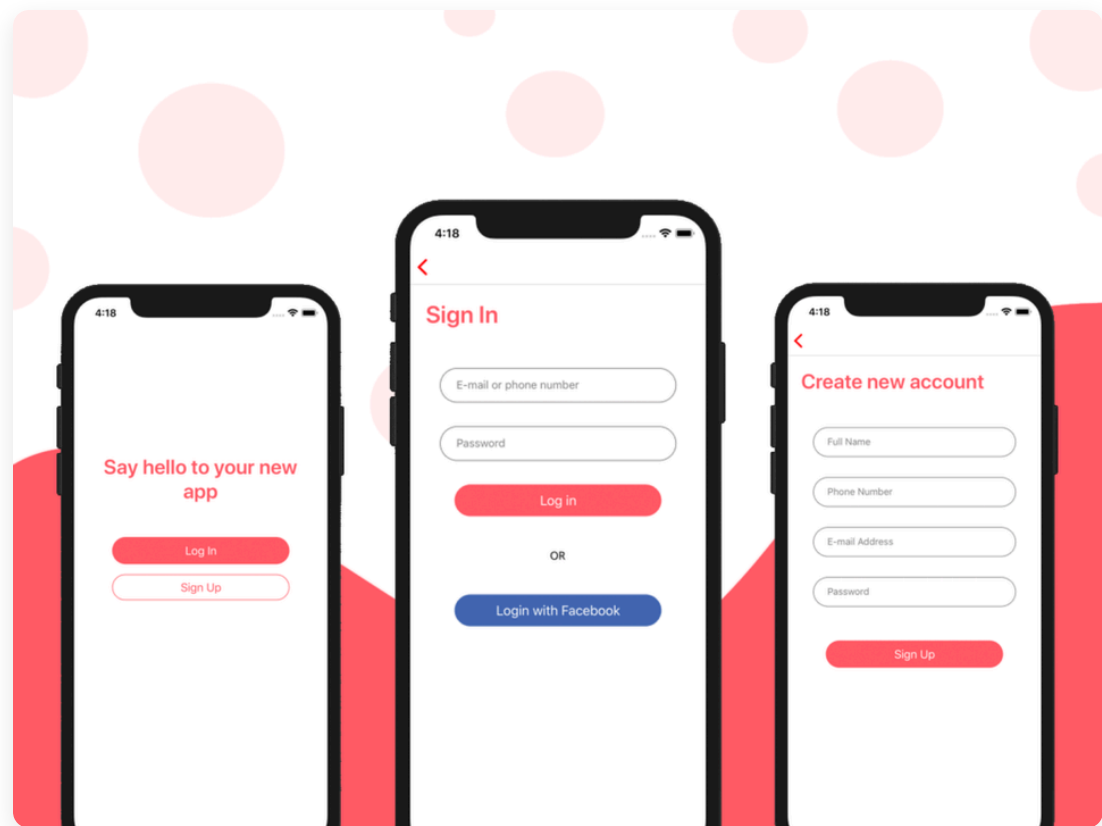
To-do List 앱의 구조와 데이터 흐름을 설계하는 방법을 배웁니다.



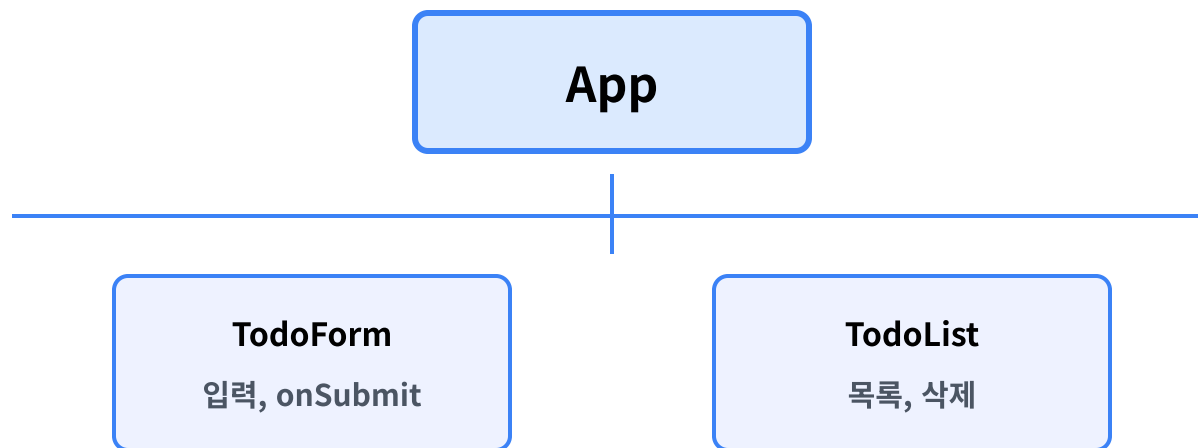
To-do List 앱 개요

- > 할 일 입력 → 추가
- > 목록 출력 (리스트 렌더링)
- > 완료 상태 토글 (체크박스)
- > 삭제 기능 포함

💡 이전에 배운 이벤트, state, form을 모두 활용하는 종합 실습 프로젝트입니다.



구성 요소 다이어그램



💡 컴포넌트 기반 설계는 React의 핵심 개념입니다. 각 컴포넌트가 독립적인 기능을 담당하여 유지보수와 재사용성을 높여줍니다.

</> 단일 책임 원칙

각 컴포넌트는 하나의 기능만 담당하여 코드의 가독성과 유지보수성을 향상시킵니다.
TodoForm은 입력과 제출만, TodoList는 목록 표시와 삭제만 담당합니다.

To-do List 데이터 구조

속성	타입	설명
id	number	각 할 일 항목의 고유 식별자로 항목 삭제 시 사용
text	string	사용자가 입력한 할 일 내용을 저장
done	boolean	할 일 완료 여부 (체크박스 상태 관리)

데이터 예시:

```
const todoItems = [  
  { id: 1, text: "React 공부하기", done: false },  
  { id: 2, text: "이벤트 핸들러 실습", done: true },  
  { id: 3, text: "To-do 앱 완성하기", done: false }  
];
```

🔗 데이터 활용

- `map()` 으로 리스트 렌더링
- `filter()` 로 항목 삭제
- 스프레드 연산자로 항목 수정

🗄️ 상태 관리

- `useState` 로 배열 관리
- 할 일 추가 시 `setTodos([...todos, newTodo])`
- 완료 상태 토글 시 `id` 기반 업데이트

단순한 객체 구조로 To-do 앱의 핵심 기능을 효과적으로 구현할 수 있습니다. 필요에 따라 `createdAt` 등의 속성을 추가할 수 있습니다.

To-do List UI 설계

- > 입력창 + 추가 버튼
- > 목록 항목: 체크박스, 텍스트, 삭제 버튼
- > 사용자 경험(UX) 중심의 직관적 인터페이스

To-do List 와이어프레임

추가

☐ 리액트 이벤트 핸들러 복습하기

☒ Controlled Form 예제 분석하기

☐ To-do List 앱 기능 구현하기

💡 UI 설계는 사용자 경험과 직결됩니다. 간결하면서도 직관적인 디자인이 중요합니다.

동작 흐름 요약

- > `onChange` 로 입력값 state 저장
- > `onSubmit` 으로 배열 state에 추가
- > `map()` 으로 리스트 렌더링
- > 삭제 버튼 클릭 시 `filter()` 로 항목 제거

To-do List 앱 데이터 흐름

입력값 감지
(onChange)

-

할 일 추가
(onSubmit)

-

목록 렌더링
(map)

-

상태 업데이트
(filter)

💡 이 흐름은 React의 **단방향 데이터 흐름** 원칙을 따르며, 모든 데이터 변화는 항상 상태(state) 업데이트를 통해 이루어집니다.

다음 차시 예고

- ✓ 오늘은 이벤트 처리와 폼의 기초를 배웠습니다
- ✓ React 이벤트 핸들링, Controlled Form 개념을 학습했습니다

4주차 2차시: To-do List 앱 구현 실습



- To-do List 앱 컴포넌트 구조 구현
- 할 일 추가/삭제 기능 개발
- 상태 관리와 이벤트 처리 실습
- 완성된 앱 스타일링 및 최적화



다음 수업을 위해 오늘 배운 **Controlled Form**과 **이벤트 핸들링** 개념을 복습해오세요!

오늘의 학습 정리 및 퀴즈 안내

오늘 배운 내용을 요약하고 이해도를 점검하는 퀴즈로 마무리합니다.



오늘 배운 핵심

</> 이벤트 처리 기초

React의 **SyntheticEvent** 시스템과 이벤트 핸들러에 함수 참조 전달 방식

⌨️ onChange / onSubmit 이벤트

입력값 변경 감지와 폼 제출 처리를 위한 핵심 이벤트 시스템

✅ Controlled Form의 장점

예측 가능한 상태 관리, 쉬운 유효성 검사, 일관된 UI 업데이트



React에서 **Controlled Form**은 더 많은 코드가 필요하지만, 앱 개발에서 필수적인 안정성과 유지보수성을 제공하는 핵심 패턴입니다.

Quiz 1

? onChange 이벤트의 역할은?

1 버튼 클릭 처리

2 입력값 변경 감지 ✓

3 DOM 직접 제어

4 컴포넌트 삭제

💡 onChange는 **입력값이 변경될 때마다** 발생하는 이벤트로, Controlled Component 구현의 핵심입니다.

Quiz 2

? onSubmit에서 e.preventDefault()를 사용하는 이유는?

☞ 정답을 선택하세요:

- 1 함수 실행 속도 증가
- 2 CSS 스타일 초기화
- 3 폼 제출 시 새로고침 방지 ✓
- 4 alert 실행 보장

! 폼 제출 기본 동작은 페이지를 새로고침하므로, **e.preventDefault()**를 사용하여 SPA에서 원치 않는 페이지 리로드를 방지합니다.

Quiz 3 - Controlled vs Uncontrolled Form 차이

- ✓ React에서 폼을 다루는 두 가지 방식의 차이점을 이해하기
- ✓ 각 방식의 장단점 및 권장 사용 사례 알기

? 📌 Controlled vs Uncontrolled Form 차이?

- 1 Controlled = DOM 직접 접근
- 2 Controlled = state 관리 ✓
- 3 Uncontrolled = 더 복잡한 코드
- 4 Controlled = 유지보수 어려움



Controlled Form은 **React state**로 입력값을 관리하고, Uncontrolled Form은 **DOM**을 직접 참조합니다. React에서는 Controlled Form을 권장합니다!

마무리 및 안내

- ✓ 이벤트 처리와 폼의 기본적인 개념을 학습했습니다
- ✓ onChange, onSubmit 이벤트를 활용한 폼 처리 방법을 익혔습니다
- ✓ Controlled Form은 React 앱의 기본 구조입니다

📅 다음 수업 안내

4주차 2차시: To-do List 앱 실습

오늘 배운 개념을 활용하여 실제 To-do List 앱을 구현해 봅니다.



오늘 배운 이벤트 처리와 폼 관리 방식은 **대부분의 React 앱에서 핵심적으로 사용되는** 개념입니다!